

Programming’s Place in Higher Education through Educator and Institutional Perspectives

Sebastian Mateos Nicolajsen*, Claus Brabrand*, Louise Meier Carlsen*

*Center for Computing Education Research

IT University of Copenhagen

Copenhagen, Denmark

Abstract—Introductory programming courses are now common across higher education, yet little is known about how educators themselves articulate the *place, purpose, and role* of programming within their programmes and institutions. We surveyed 146 educators (N=81 responses) across 175 mandatory introductory programming courses in Denmark, spanning computer science, engineering, and non-computing disciplines, and analyzed their written reflections using an inductive approach. This study explores why such courses are created, what they are intended to achieve, and how programming is positioned within broader disciplinary trajectories. From this analysis, three working hypotheses emerge: (H1) that introductory programming courses are primarily established to address (*bottom-up*) *competence gaps* among incoming students rather than to fulfill (*top-down*) *curricular visions*; (H2) that programming is generally perceived as a *preparatory enabler* for later courses rather than a standalone disciplinary subject; and (H3) that programming is most often framed as a *pragmatic tool* which augments existing methods and practices, for example in data analysis, statistics, and scripting. Read through an institutional lens, these findings provide the first systematic evidence that institutions are highly pragmatic in how they position programming—prioritizing functional integration and disciplinary utility over abstract ideals such as algorithmic or computational thinking. The results contribute a grounded account of programming’s institutional rationale and offer practical insights for computing-education researchers, curriculum designers, and educators seeking to align disciplinary needs, institutional priorities, and effective programming education.

I. INTRODUCTION

There are long-standing viewpoints on the role of ‘computing’ and ‘programming’ in education, whether it be the teaching of programming [1] or computational thinking [2]. At the same time, there is a call for broader and better inclusion of computing in education [3]–[6]. However, little to no investigation has been made into what the *educators* of courses perceive ‘programming’ and ‘computing’ to be in *their* local educational context, whether inside or outside of computer science (or engineering).

If programming is to be meaningfully positioned in curricula, we must consider how it is perceived by the *educators*, which have a critical role in transposing curricula to teaching/learning activities [7] [8] [9]. In this study, we also examine, through educator statements, how programme- and institution-level rationales shape why and how introductory programming courses are positioned in curricula. Because course content, sequencing, and assessment are negotiated within programmes

and institutions, educator accounts inevitably reflect institutional priorities alongside personal views. We hope this research will nuance the debate further and shed light on some of the factors affecting the role of computing education. As stated by Vogel et al.: “*we are currently setting sail without a clear sense of where we are going*” [10].

To this end, our study focuses on *educators* involved in teaching *mandatory introductory programming* in all higher education disciplines (computer science as well as non-computer science programmes). We explore what Danish introductory programming *educators* perceive ‘programming’ to be and what *role* it plays within their local educational programme. In particular, we investigate how educators perceive ‘programming’ in relation to their course and educational programme through the following three research questions (RQ1–RQ3):

RQ1 (Course Creation Reason): *What are the reasons for creating programming courses?*

RQ2 (Course Purpose): *What is the purpose of the course introducing programming?*

RQ3 (Programming Role): *How do educators perceive the role of programming in their programme?*

By exploring these research questions, we aim to provide an evidence-based insight for future research on the topic of programming course positioning in diverse disciplinary areas.

To clarify our analytical frame, we distinguish three complementary levels of reasoning about programming courses:

- **Reason for Course Creation (RQ1):** Why the course came to exist, often reflecting institutional needs or gaps in student competences;
- **Course Purpose (RQ2):** What the course is intended to achieve within the programme sequence;
- **Programming Role (RQ3):** Why programming is viewed as belonging to the discipline or profession.

Read together, these levels jointly describe *programming’s place* in higher education—its institutional, didactical, and disciplinary positioning. Further, we hope to explicit the role of the institution in such positioning.

From these distinctions, we derive three working hypotheses—each corresponding to a dominant institutional logic observed across levels of analysis:

In concert, **RQ1 → RQ2 → RQ3** constitute **Programming’s Place** in Higher Education:

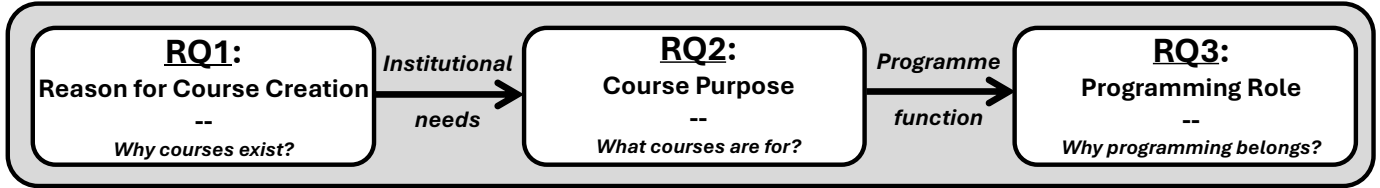


Fig. 1. Conceptual progression of the study’s three research questions (RQ1–RQ3): How institutional creation (RQ1), programme purpose (RQ2), and disciplinary role (RQ3) jointly define programming’s *place* in higher education.

- **H1 (Creation Logic):** Institutional introductory programming course creation is driven by (*bottom–up*) emerging demands rather than (*top–down*) imposed visions.
- **H2 (Programme Logic):** Programming is positioned as a *programme-level enabler* that supports subsequent courses rather than a stand-alone *disciplinary subject*.
- **H3 (Disciplinary Logic):** Educators conceptualise programming primarily through *pragmatics* rather than *abstract or cognitive framings* (from research).

These three institutional logics—creation, programme, and disciplinary—serve as analytical reference points that may be supported, nuanced, or contradicted by the inductive results that follow.

II. BACKGROUND AND RELATED WORK

In general, studies exploring *educator viewpoints* focus on struggles and challenges associated with teaching; e.g., teacher knowledge, competences, or pedagogical approaches [11]–[13]. A few studies, however, explore educator viewpoints in computing education. Vogel et al., for example, examined how various New York City-based stakeholders envisioned computing education through the lens of seven societal and institutional areas [10]. Besides this, Greifenstein et al. found that primary school teachers (including those in training) perceive the opportunities of programming to be *skill acquisition*; i.e., problem-solving and logical thinking, linguistic stimulation, creativity, digital literacy, computational literacy, reduction of prejudices, interest, and motivation [14]. This is further reinforced by a literature review by Saeli et al. [15]. Greifenstein et al. also highlight computings function in primary school to support unspecified ‘future endeavors.’ Elaborating on this, Korhonen et al. argue that Finnish primary school teachers generally associate ‘programming’ with ‘*important skills in society*’, which, in turn, may support students in developing ‘*other skills*’ [16]. Others, such as Medeiros et al., argue a need to further clarify some of these components; e.g., *problem-solving* [17]. Generally, ICT and computation is argued to support societal preparedness and general academic skills [18], [19].

While prior work has proposed taxonomies of teacher views on programming (e.g., Pörn et al. [20]), we retain it here as a comparative lens rather than as an organizing framework. In brief, Pörn et al. classify teacher responses into six views:

programming as stepwise instructions (sequential), as logical/patterned reasoning (logical), as algorithms and abstraction (algorithmic), as problem-solving, as understanding technology and digitalization (technological), and as preparation for future work (progressional).

For higher education, we have previously conducted a comprehensive analysis of more than a thousand higher-education programmes and their course descriptions, identifying almost two hundred programmes that include mandatory programming introductions [21]. Their study highlights how programming is incorporated across a wide range of disciplinary areas, reflecting the specific needs and contexts of each discipline. Their findings suggest that the most emphasized programming competences (across disciplines) are related to *problem-solving* (through *design, implementation, and validation*) rather than *abstraction & algorithms* (which are often at the foreground of computer science research). They also argue that, for non-computer science disciplines, educators ought to prioritize *problem-solving* as a central competence when introducing programming, and consider what this means within their own fields. Finally, they underline the need for educators to take *ownership* of what introducing programming entails and how it should be presented, supporting the need for a study which explores educator viewpoints.

Within the more general sphere of computing education research, we find a series of topics related to viewpoints on programming education. For example, the introduction of *computational thinking* in 1980 by Papert, related to the expected thinking capabilities established through use of the programming language LOGO¹ [22]. Since then, the concept of computational thinking has become a significant envisioning of computing/programming, with a variety of different interpretations, some following the original thoughts by emphasizing the importance of programming as a *computational literacy* [23], [24]. Others focus on the *problem-solving* capabilities arising from learning computing [2]. In particular, the later re-envisioning by Wing (in 2006) sparked a significant movement towards “computing for all” [2]. Although contemporary *computational thinking* has a range of different meanings, it broadly envisions skills learned through computer science that may be applied in “*other contexts*” and recognizing aspects of

¹A “turtle graphics” programming language for controlling a turtle with two fundamental operations: *moving forward* (effectively drawing) and *turning* (e.g., 90° clockwise).

computation in the world [25]–[30]. Similarly, political ideas such as *digital competences/digital literacy* reflect a collective belief and an intention to promote digital competences in education, defined as “a multifaceted moving target, covering many areas and literacies and rapidly evolving as new technologies appear” [31]–[33]. Shared among these ideas is a repertoire of skills, such as *abstraction*, *decomposition*, *algorithmic thinking*, and a *problem-solving* approach [2], [33]. In our analysis, we reference such constructs only as points of comparison while foregrounding inductively derived educator themes.

The use of computing also effects a perception of it. Ko et al. argue that computing is more than a tool for *expression*: It should also consider the *social* and *cultural* impact on development [34]–[36]. Evrard and Guzdial continue this work by describing alternate reasons to teach computing; i.e., for exploring rich systems composed of models (simulation) and for understanding how it affects *culture*, *society*, and *politics* [37], [38].

In conclusion, there has been *some* effort in relation to (and in particular) primary school perception of programming, its role, and its purpose. In regards to higher education, this is limited to document analysis of existing courses. Further, the general computing education introduces a series of different viewpoints, which drive the computing and programming education today. Yet, there is currently not much research investigating the viewpoints of the wider educators space within programming education. Thus, this study fills this gap, by investigating how educators (and indirectly their institutions) view programming.

III. METHODOLOGY

We investigated how *educators* perceive ‘programming’ in the context of Danish higher education programmes. The limited size of the country (approximately 5.9 million inhabitants) allows us to cover the entirety of a nation (Denmark) and thus gain a comprehensive national broad range of voices. Specifically, we investigated a total of 1,169 distinct educational programmes aggregated using the official public list of Danish educational programmes.² We focused on *Bachelor*, *Master*, and *Professional/Engineering Bachelor* degrees since educational institutions in Denmark are free to define the curricula themselves for such programmes. The aggregated data and an analysis of it are based on our existing study [21].

From this, we compiled a list of educators who teach mandatory courses introducing programming. Courses/programmes were included if and only if a course *title*, *description*, or *curriculum* contained programming-related words (*programming*, *coding*, *scripting*) or mentioned *software* that involves programming or referenced a specific programming *language* (e.g., *R* or *Python*). This led to the identification of 130 courses (spanning 175 educational programmes) and a total of 146 educators (including both

those who teach as well as those who are responsible for courses). Although this limits identification of some programmes, it does capture various courses such as statistics applying programming software. We focused solely on courses which were *mandatory* in educational programmes. Despite the size of the data, a wide array of different disciplines and higher-education educators are represented in this population.

We then designed and distributed a survey to *all* (146) educators. The survey contained *five questions* (Q1–Q5), detailed in Figure 2, related to the aforementioned research questions (RQ1–RQ3). In addition to the five questions, the survey also included background questions which we will use to characterize the educator population (see Section IV) and other questions which are beyond the scope of this study of educator viewpoints on programming. The survey was designed following Rowley’s guide to questionnaires [39] and was designed to take different (educational) *roles* into consideration; in particular, *teachers*, *course responsables*, and *heads of study programmes/programme directors*. Hence, not all questions were presented to all roles; e.g., only course responsables (part of creating a course) were asked about reasons for its creation. Because of this, we provide the number of respondents for each question (which is thus invariably not the same for all questions). Note also that the questions were not necessarily presented in the same consecutive order (Q1–Q5). Due to the role-sensitive complexity of the survey, we conducted several pilot interviews with colleagues and a pilot execution involving three individuals (one for each role) to assert the quality of the survey.

To analyze the responses, we deliberately applied an *inductive coding approach* to avoid mis-categorising responses as well-known ideas from the CER community (e.g., *computational thinking* or the underlying skills such as *abstraction* or *decomposition*) if not mentioned explicitly, so as to ensure that we reflect the viewpoints of *educators* (ideally without pollution from CER community preconceptions). While we here introduce three hypotheses regarding the logic of such courses, these are not bound to specific framings or concepts within computing education (as is customary for movements such as computational thinking). Further, through working with hypotheses we allow ourselves to disprove them, in case the data tells a different story. The first author (C1) inductively coded the questions individually to identify themes (arguments) and devised a coding guide from this. A second coder (C2) coded the data using the designed guide. Matching the codes identified by the first (C1) vs second coder (C2) for each question individually, we obtain IRR (Inter-Rater Reliability) agreements in the range of 89% to 98%, with a Kappa between 0.85 and 0.97, implying *substantial agreement* between the two coders, for all five questions [40]. We later report the themes which appeared the most, however, our coding also resulted in themes which were not included in the codebook passed to the second coder, e.g.; *Communication*, *Intended learning outcomes*, *Learning tools*, and *Computing potential*. A full overview of the codes included in the codebook is

²The National Danish Education Guide: <http://ug.dk>

Research Question (RQ):	Q:	Question:
Course Creation (RQ1):	Q1:	Which conditions led to creating the course?
Course Purpose (RQ2):	Q2A:	What is the purpose of the course?
	Q2B:	What is the course's relation to the overarching goals of the educational programme(s) it is part of?
Programming Role (RQ3):	Q3A:	Why, in your opinion, is programming part of the course?
	Q3B:	What value does programming bring to the education; e.g., in terms of acquired skills or knowledge?

Fig. 2. The five questions of the survey (translated from Danish) that the educators were asked to answer (cf. RQ1–RQ3).

provided in Table I.

Code	Description/Example
Demand / Increased relevance	Programming became a requirement, e.g., new skills/competence needed (often for course creation)
Programming in general	Is the goal to teach programming constructs in general?
Specific programming language	Is the goal to teach a specific language, e.g., Python, Java?
Tool	Is the goal to teach a tool, e.g., MATLAB, R?
Work with data	Is the goal to teach how to analyse datasets?
Statistics	Is the goal to teach statistical methods?
Computational thinking competences	Is the goal to teach problem-solving, modelling, reasoning, abstraction, etc.?
Preparation for future endeavors	Is the answer concerning preparing students for future work/study?
Programming as a tool	Is the answer concerning how programming augments or enables new methods? (e.g., “programming augments or enables new methods”)
Self-evidence	Is the answer concerning programming being included as self-evident necessity? (e.g., “simply because of programming”)
Provide insights into discipline	Is the answer concerning giving students insight into the discipline? (e.g., “to give students insight into the discipline”)

TABLE I

SIMPLIFIED ‘CODEBOOK’ FOR CATEGORIZING EDUCATOR RESPONSES.

IV. POPULATION

We now characterize the population of introductory programming *educators* who responded to our survey distributed to 146 educators. We received a total of $N=81$ responses (hereof 13 partial responses), yielding a response rate of *more than half* (55%). Among these educators: 26% *teachers*, 67% *course responsables*, and 7% *others* (including 4 heads of study programmes/programme directors); hereof 67% self-identified as *male*, 32% as *female*, and 1% as *non-binary*. Since the survey was about programming *beyond* computer science, we, unsurprisingly, see a more balanced 2:1 gender diversity (compared to the 4:1 ratio associated with ICT in Europe) [41]. Although the majority of educators are of Danish nationality (60%), 23% are from broader Europe, and 17% from the rest of the world. The foreign educators responding

to the survey have been employed in Denmark for an average of 8.1 years and, therefore, have extensive experience with the Danish educational system. In terms of seniority: 15% are Full Professors, 62% are Associate Professors, 18% are Assistant Professors, and 5% are PhD students, Research Assistants, or equivalent. The educators come from a variety of different backgrounds: a third (36%) are exclusively affiliated with computer science (or engineering) departments; half (49%) are not affiliated with a computer science department; and 15% are affiliated with both a computer science as well as some non-computer science department. In terms of programming expertise: the majority (87%) of the educators know how to program; and, among these: half (52%) learned through *formal* education, 36% are self-taught via *informal* education, and 12% through a mix of *formal* and *informal* education. Some effectively trained themselves, driven by *self-perceived demand* (or, simply the pressure to “stay afloat”):

[I was] drowning in data as a PhD student. It was “sink or swim”—a not-so-useful biostatistics course, some helpful colleagues, and a lot of googling got me started.

A. Disciplinary Areas

We use the *governmental educational register* of Denmark [42], which categorises educational programmes unilaterally according to their so-called “disciplinary domain,” featuring broad categories such as *Natural Science* or *Humanities*. Altogether, the responses cover 62% of the identified programmes within each disciplinary area (with at least one educator responding for each course³). Except for the joint disciplinary area ‘*Business Economics, Administration, & Law*,’ almost half (or more) of the solicited educators responded. Table II provides a comprehensive overview of responding educators according to disciplinary area. Table IV provides a comprehensive overview of the distribution of the 175 educational programmes with mandatory programming according to their disciplinary area (out of all the 1,169 educational programmes).

B. Educational Institutions

As for educational institutions, the identified educational programmes span 14 higher-education institutions; we received responses from educators from about two-thirds of the

³Please note that courses can be shared among educational programmes.

Disciplinary area	Coverage
Social & Health	100% (1/1)
Natural Sciences	76% (34/45)
Agriculture, Forestry, & Fishing	75% (6/8)
Building & Construction Engineering	67% (2/3)
Mechanics, Iron, & Metal	63% (5/8)
Information, Communication, & Technology (ICT)	66% (25/38)
Engineering, Technology, & Industrial Production	48% (28/58)
Social Science	44% (4/9)
Business Economics, Administration, & Law	20% (1/5)

TABLE II

THE NUMBER OF EDUCATIONAL PROGRAMMES IDENTIFIED TO HAVE INTRODUCTORY PROGRAMMING FOR WHICH *at least* ONE EDUCATOR HAS RESPONDED. CATEGORISED ACCORDING TO DISCIPLINE.

institutions (9 of 14). Except for one, all Danish universities are represented (7 out of 8). Among the larger institutions with more than a handful of educational programmes introducing mandatory programming, half (or more) of the educators responded. Table III provides an overview of educator respondents according to educational institution.

TLA	Institution	Coverage
ITU	IT University of Copenhagen	100% (7/7)
KU	University of Copenhagen	81% (22/27)
AU	University of Aarhus	72% (21/29)
SDU	University of Southern Denmark	58% (19/33)
DTU	Technical University of Denmark	54% (13/24)
AAU	University of Aalborg	50% (20/40)
EAS	Business Academy Southwest	50% (1/2)
RUC	University of Roskilde	40% (2/5)
KEA	Copenhagen School of Design & Tech'	33% (1/3)
CBS	Copenhagen Business School	0% (0/5)
UCA	University College Absalon	0% (0/1)
VIA	VIA University College	0% (0/1)
EAD	Dania Academy	0% (0/1)
IBA	International Business Academy	0% (0/1)

TABLE III

THE NUMBER OF EDUCATIONAL PROGRAMMES IDENTIFIED TO HAVE INTRODUCTORY PROGRAMMING FOR WHICH *at least* ONE EDUCATOR HAS RESPONDED. CATEGORISED ACCORDING TO INSTITUTION. TLA IS A (THREE-LETTER ACRONYM) INSTITUTIONAL ABBREVIATION.

V. RESULTS

We now consider the results for each of the three research questions and then use the results to approach how educators perceive 'programming' the *role* of programming within their courses and educational programmes. Some answers (e.g., "the purpose is to teach programming") appear across multiple questions because respondents articulate overlapping concerns. Nonetheless, each research question targets a different level of reasoning: RQ1 (Course Creation) asks why courses exist (institutional origins and competence gaps); RQ2 (Course Purpose) asks what the course is for within the programme; and RQ3 (Programming Role) asks how programming is positioned in the discipline/profession. We therefore report results separately to respect these distinctions, and later synthesize across questions in our discussion. Before detailing each research question, we remind the reader of the conceptual distinctions:

Disciplinary area	Coverage
Mechanics, Iron, & Metal	53% (8/15)
Natural Science	42% (45/107)
Information & Communication Technology (ICT)	41% (38/91)
Engineering, Technology & Industrial Production	29% (58/198)
Agriculture, Forestry, & Fishing	22% (8/36)
Building & Construction	10% (3/30)
Social Science	8% (9/108)
Business Economics, Administration, & Law	3% (5/138)
Social & Health	1% (1/90)
Humanities	0% (0/163)
Teaching & Learning	0% (0/53)
Artistic	0% (0/53)
Service	0% (0/15)
Audio-Visual Technology & Media Production	0% (0/14)
Transportation	0% (0/4)
N/A domain	0% (0/4)
Defence, Police, & Security	0% (0/1)

TABLE IV

DISTRIBUTION OF EDUCATIONAL PROGRAMMES *with* MANDATORY PROGRAMMING *vs* THOSE WITHOUT.⁴

- **RQ1 (Course Creation Reason):** Addresses the *origin* of the course—its creation and institutional justification.
- **RQ2 (Course Purpose):** Concerns the *function* of the course within its programme—its immediate pedagogical purpose.
- **RQ3 (Programming Role):** Explores the *disciplinary rationale*—the role programming is perceived to play in the field.

Each level thus speaks to a different dimension of programming's broader positioning in higher education. All responses are kept anonymous for the sake of our participants.

A. What are the reasons for creating programming courses? (RQ1)

This first (RQ1) question targets the *origin* of programming courses—why they were created and what institutional or competence-related needs they were intended to address. When asking those responsible for courses who also took part in *creating* the course (26 educators) about the conditions that led to the creation of the course (Q1 in Figure 2), 73% (19 of 26) argued that it was due to *demand* and a change in the student *competences needed*. While the majority of answers simply state an (unelaborated) "*need for competences*." In some cases, the necessity of a course became only fully evident *after* its creation:

Upon my returning to [my institution], I could see that there was no [R]-course, so I decided to create it. First, as a pilot in 2018, and [later as a fully-fledged course]. I have created the entirety of the course and I am the sole responsible and sole teacher and executor of the course. Evidently, there was a gap in the course catalogue and the course went from ~35 to ~150 students.

Some departments implemented *reforms* to ensure programming competences across their programmes. However, this

⁴The dataset originates from this study [21]

required rethinking of (in particular) the *specific* programming language used:

Previously, introduction to Java programming was mandatory for many Science BSc educations—[other] educations did not want to continue having Java as a supplementary course. Math demanded a Python course, which evolved into the current course also being mandatory for Math-Economics and more recently [Data] Science. Essentially all chemistry students also have it as an elective course.

Further, it is worth noting that educator arguments are rooted in their own disciplinary needs and local contexts. In summary:

OBSERVATION 1: *Educators largely argue that their courses were created & introduced to ensure students acquiring competences they were previously lacking: programming, in general, or knowledge of a specific programming language; e.g. R or Python.*

B. What is the purpose of the course introducing programming? (RQ2)

For RQ2, we now turn from origins to *function*: What educators perceive as the immediate pedagogical purpose of their course within the programme sequence. In Q2A, 67 educators commented on the *purpose* of the course. Hereof, 43% (29 educators) argued that the purpose of the course was to teach (general) programming:

To learn general procedural and object-oriented programming techniques, and be able to combine them to a program that can make the computer help execute the task at hand.

However, educators often did not mention programming as the *sole purpose*; some, for instance, briefly touched upon attributes typically ascribed to computational thinking:

To teach programming, abstract and concise thinking, and bioinformatics.

While educators largely comment on learning programming independent of a language, 19% (12 educators) highlighted the teaching of *a specific* programming language; for example: “*learn to program in C*,” “*to teach Python programming*,” or “*to teach people with no programming experience how to write simple programs in Java*.”

Beyond programming, 19% (13 educators) focused specifically on student ability to *work with data* as a means to support (unspecified) future endeavours within their domain:

Furthermore, the aim is to provide students with the necessary methodological and data analysis skills to be able to evaluate validity and quality of methods and data related to analysis of large datasets.

Even more specific to data, 15% (10 educators) argued for a need for competences related to *statistics*:

To teach biology students programming and statistics that they can use in the analysis of biological data.

In summary:

OBSERVATION 2A: *Educators largely argued that the purpose of their course was to teach fundamentals of programming, and to support students with tasks within and outside of computing, drawing on competences associated with computational thinking and statistics.*

In Q2B, when asking educators about the purpose of their course *in relation to the overarching goals of the educational programme*, we received 19 responses. Supporting our initial findings, 63% (12 educators) argued that the goal of their course is to *support students for later courses* (in the programme): “*It is a fundamental course for students to be able to continue with the other courses in the programme*.” Although one could expect the above argument to be made exclusively by computer science programmes, this is only the case for 7 of them; the remaining span everything from *health technology* to *mechanical engineering*. Further, some argue that introducing programming can translate to general insights regarding computing and thereby empower students:

For most educational programmes including this course, the course is an isolated course in their programme, and many will not be exposed to other dedicated programming/computer science courses [...]. For most students, programming is limited to using some tools in their major curriculum. This course should empower the students with general skills to independently go deeper into these areas if needed.

In summary, programming as an indispensable prerequisite:

OBSERVATION 2B: *Educators largely argue that the goal of their course is to support students for future courses, later in the educational programme.*

C. How do educators perceive the role of programming in their programme? (RQ3)

Finally, for RQ3, we address the *disciplinary rationale*—why programming is seen as belonging to the field, including what epistemic or professional role it plays. Although educators may argue that the introduction of programming and the course in itself have the same purpose, asking educators specifically about their *opinion on why programming is part of the course* may elicit other perspectives. We got 45 answers to Q3A.

Although the answers largely cover the same topics as previously identified, 35% (16 educators) argued for programming to be a useful, almost crucial, *tool* in their discipline; e.g., “*because statistics of large data sets cannot be run with a GUI*” or “*because math without programming gets us nowhere*.” Broadly, these answers appear to express an *augmentative effect* of computing in relation to disciplinary methods and tools.

To our surprise, 20% (9 educators) simply replied (vaguely) that “[*programming*] is an *essential element*” (or variations thereof). This largely supports the ubiquitous necessity of computing/programming:

OBSERVATION 3A: *Many educators designated programming as an essential element that provides an augmentative effect for their disciplinary methods and tools.*

In Q3B, we asked educators *what value programming brings to their educational programmes*, which resulted in 41 responses. The educators provided three different arguments towards its purpose.

First, 53% (22 educators) reiterated programming’s purpose *as a tool* and its *impact* on conducting their tasks, supporting the idea of conversational or vernacular programmers [37], [43]:

It is a must for all natural science students (and others) to be able to handle data and make quantitative assessments of data. In this regard, programming is a necessary tool now and in the future.

Not only is it a crucial tool, but it allows students to address challenges in a new and creative way; again *augmenting existing methods* within their discipline:

It allows users to be creative in their way to address a challenge. Say, a large dataset that is a bit complicated to analyze. Basic statistic software cannot always work to make the correct comparisons, or make suitable figures. Programming can allow you to bring additional resources to address these challenges.

Second, 22% (9 educators) argued for *general-purpose thinking skills*, which they describe as obtainable through programming, similar to what is advocated through computational thinking [2], [44], [45]:

Many students don’t know anything about programming. Many student are not really good with a computer in any way beyond starting up Word. Once you experience programming, you start to think in a different way, and other useful tools and techniques become much easier to get started with. For example: LaTeX, scripting in CAD-programs, configuration of complex software systems.

Third, 14% (6 educators) do not necessarily see an explicit purpose of programming, but instead turn towards the (relevant) *insights into the computing domain* afforded by it (almost as a bi-product):

General knowledge about software (someone implemented it) and use for specific purposes. / Giving people who are going to be consultants or project managers an ideas about what programming is.

While programming for a few (9 respondents) aligns with computational thinking, educators largely focus on different arguments, namely the role computing already has in their own discipline. We summarize as follows:

OBSERVATION 3B: *Educators mention that the value of programming is as: (1) a tool that impacts the tasks of their discipline; (2) general-purpose “thinking skills”; and (3) providing insights into the computing domain.*

VI. DISCUSSION

We interpret and discuss the results presented above from two angles. Before doing so, it is useful to recall that our three research questions form a progression—from institutional origins (*reason for creation* in RQ1), to pedagogical function (*course purpose* in RQ2), to disciplinary rationale (*programming role* in RQ3). Together they reveal programming’s *place* in higher education as an emergent synthesis of these dimensions. When revisiting the analysis through these hypotheses, we find that the evidence broadly supports all three institutional logics: (H1) course creation is shaped by bottom-up demands rather than top-down design; (H2) programming is framed as a programme-level enabler rather than a stand-alone subject; and (H3) educators’ disciplinary perspectives emphasize pragmatic relevance over abstract or cognitive framings. First, we suggest an institutional reading of the results. Second, we situate and synthesize these findings relative to prior frames (e.g., Nicolajsen et al.; Pörn et al.) to highlight both overlaps and differences, and how these findings may be used going forward [20], [21].

A. Institutional reading

By an institutional reading, we attend to a course’s *raison d’être* (its reason for being): why it exists, how it is positioned in a programme, and what functions it is expected to serve. The notion of *raison d’être* is articulated explicitly in the Anthropological Theory of the Didactic (ATD) e.g., [46], [47], a mathematics education research tradition that examines relations between institutions, knowledge, teaching, and learning. Although ATD may be less familiar within engineering and computer science education, it is useful here because it makes institutional justifications explicit.

Through the concept of didactic transposition, the focus shifts to the processes by which scholarly and professional knowledge is transformed as it moves into educational institutions. This includes the selection of knowledge from reference practices, its reorganization into curriculum objects, and its adaptation into teachable activities. Didactical transposition thus draws attention to the institutional rationales underlying curricular choices: why certain content is retained, modified, or excluded; how knowledge is structured and sequenced across courses; and how these decisions reflect broader disciplinary traditions, professional expectations, and institutional constraints. In our analysis, we therefore read educators’ survey responses as expressions of institutional rationales rather than individuals’ opinions. We examine: (i) why the course exists, (ii) where it sits within the programme sequence, and (iii) what functions it is expected to fulfil for students and the discipline. Concretely, this yields:

- **RQ1 (Course Creation)** reflects institutional needs and competence demands (e.g., coverage gaps, shifts in required tools/languages, stakeholder expectations).
- **RQ2 (Course Purpose)** reflects the prescribed function of the course within the programme (e.g., preparing students for later courses, enabling data work).

- **RQ3 (Role of Programming)** reflects disciplinary/-professional trajectories shaped by the institution (e.g., programming as a tool indispensable for domain tasks, broader readiness to engage with digital systems).

Emerging from the responses, we also see other themes: competence gaps among incoming students; preparation for later courses; programming as a disciplinary tool that augments existing methods (e.g., data analysis, scripting); and broader gains (e.g., working with digital systems, confidence using technical tools). Whether a course emphasizes learning to program, a specific language, or data-centric work, respondents consistently frame programming as practical, consequential for succeeding elsewhere in the programme, and valuable beyond the single course instance.

Notably, some responses present the inclusion of programming as almost self-evident, offering little explicit justification. We interpret this not as a lack of reflection, but as an indication of how deeply programming has become embedded in contemporary disciplinary practices and institutional expectations. In such contexts, programming no longer requires explicit legitimization; its presence is taken for granted.

Taken together, these responses indicate that institutions appear quite pragmatic in how they position programming. Within a didactical transposition perspective, programming courses emerge as a transposed curricular object whose continued presence is justified by its capacity to meet institutional and disciplinary needs, addressing competence gaps, preparing students for later courses, and providing students with tools for disciplinary practice, rather than by adhering to abstract educational ideals. While this aligns with preconceived notions in the wider community, there has until now been no evidence substantiating this view. Our findings, therefore, provide a first systematic account that makes explicit how programme-level priorities shape the role of programming.

Implications. Beyond theoretical relevance, the results hold practical value for multiple stakeholders:

- **Computing-education researchers** can use these findings as a grounded reference when formulating theories of programming education across disciplines, helping to align constructs such as *computational thinking* and *disciplinary competence* with the realities faced by educators.
- **Curriculum designers** may apply the results to audit whether local curricula balance pragmatic programme needs with broader educational aims, identifying where programming courses act as service elements versus integrative components.
- **Educators beyond Computer Science** can use the findings to benchmark their own course rationales, reflect on whether programming is merely preparatory or could be deepened into a more disciplinary role, and draw language for articulating such aims to colleagues and management.

In concert, these implications illustrate how evidence about educators' perspectives can inform both policy and practice,

strengthening the bridge between research, teaching practice, and institutional strategy.

B. Existing viewpoints in computing

When considering the curriculum analysis we previously conducted, i.e., [21], wherein we argue that educators should reflect on what programming can offer their discipline, how it should be integrated, and which competences should be prioritized, we can support this, in that educators largely focus on near-term competence for later courses. As a result, their reasoning is often implicit and shaped by practical concerns rather than explicit engagement with broader policy questions. This pattern is consistent with the institutional reading above (course positioning and sequencing within programmes). Relative to, for example, Pörn's six-view taxonomy [20], the accounts we observe align more with the pragmatic, problem-oriented and preparation-for-work facets.

A few responses explicitly mention general-purpose thinking skills commonly associated with computational thinking (CT) [2], typically grounded in practical applications (e.g., facility with complex software, scripting, or technical configuration). We also see occasional references to broader uses of computing beyond expression [34], [37]. Overall, however, the dominant themes remain pragmatic and programme-oriented rather than explicitly CT-framed. Thus, few responses explicitly deploy the vocabulary associated with computational thinking (e.g., decomposition, algorithmic thinking), even when adjacent practices are present. There may be multiple explanations for this disconnect: Educators may, for instance, not be using particular terminology explicitly, *or* educators may assume it to be implicitly incorporated through terms such as 'skills', *or* educators are perhaps unaware of the viewpoints of the computing education research community. This points to an important role for researchers: rather than expecting educators to take on the additional burden of curriculum reformation in accordance with research output, researchers should seek to understand the perspectives and constraints of educators themselves. By grounding broader frameworks, as is the case with our previous work, in the lived realities of teaching, researchers can help ensure that recommendations and policies are both relevant and feasible. Given that educators operate within programme structures, accreditation demands, and resource constraints, their accounts are best read as jointly personal and institutional.

In general, this underlines the conclusion of our institutional reading: building prerequisites and enabling downstream courses and disciplinary practices is the central goal. We, therefore, suggest that future courses should be focused on competences of the individual disciplines. For example, educators promote particular skills, such as the ability to apply computational problem-solving in the context of data sets (*narrow roles*), rather than applying broader concepts such as *abstraction & decomposition (broad roles)* which currently permeate the research community.

VII. LIMITATIONS

Our findings should be read in light of several limitations. First, institutional logics are inferred indirectly from educator accounts and may blend personal and organizational perspectives. While this may seem a weakness, we consider it a strength: educators are embedded within institutions, and their viewpoints naturally express both personal beliefs and systemic constraints. Second, the study is confined to one national context which, while broad in disciplinary scope, reflects governance structures and industry ties specific to Denmark. In particular, Danish higher-educational institutions operate under strong external governance, programme autonomy, and close collaboration with industry, which may not generalize to all contexts. Third, survey responses were concise, and although the high inter-rater reliability of our coding suggests thematic saturation, longer qualitative interviews might have revealed richer rationales, tensions, and internal disagreements among educators. Fourth, institutional priorities, resource constraints, and accreditation frameworks likely shape how educators articulate programming's role; these may differ in systems with weaker external accountability or greater curricular freedom.

Fifth, some disciplinary areas wherein programming education is mandatory, did not respond to the survey (e.g., humanities). While this may impact the generalizability of the findings, we note that several of the other disciplinary areas cover some similar programmes as the disciplinary categorization is unilateral, i.e., only allows a single categorization. For more on this, we refer to our previous publication using the same data set [21]. Finally, while we deliberately avoided imposing pre-defined CER terminology, this also limits comparability: future work could trace whether differences in language between researchers and educators reflect conceptual divergence or simply disciplinary translation.

VIII. CONCLUSION

The computing-education research community aims to improve the learning of computing. We argue that this requires understanding what educators themselves perceive programming to be in *their* educational context, since they are ultimately responsible for teaching it.

This study investigated educators across higher-education programmes that teach programming, exploring: (1) reasons for *creating* courses, (2) their *purpose*, and (3) the *role* of programming. We found that courses are typically introduced to address missing student competences (RQ1), positioned as prerequisites for later courses (RQ2), and valued for augmenting disciplinary methods and digital confidence (RQ3). In concert, these findings substantiate three institutional logics: H1 (Creation Logic), H2 (Programme Logic), and H3 (Disciplinary Logic)—together describing how institutional, curricular, and disciplinary perspectives shape programming's place in higher education.

In practical terms, our results can assist:

- **Computing-education researchers** in grounding theoretical frameworks in educator realities;

- **Curriculum designers** in aligning curricular intent and sequencing with on-the-ground rationales; *and*
- **Non-CS educators** in positioning programming more deliberately within their disciplines.

Read through our institutional lens, introductory programming functions as an enabling, service-like course aligned with programme-level priorities across CS and non-CS contexts. Our contribution is to provide systematic evidence that institutions are largely pragmatic: courses persist because they meet concrete programme needs rather than abstract ideals. While this pragmatism partly aligns with existing assumptions, it had not previously been evidenced.

To strengthen alignment between institutional aims and educator practice, we advocate a *competence-pragmatic approach* to programming education. The first step concerns understanding what programming is expected to accomplish in context. Rather than assuming universal goals, institutions and educators must first clarify the competences programming should develop for their particular programmes and disciplines (which may vary a great deal between disciplines):

RECOMMENDATION 1 (Identify Need): Effective course design begins with recognising *what competences are actually needed*. Identify the specific computing and programming competences required within each disciplinary and institutional context, acknowledging bottom-up demands and local sequencing. This should guide course creation so programming fulfills concrete and pragmatic needs rather than top-down abstract mandates and visions.

Having identified such needs, the next step is to ensure that educators are supported in realizing them in practice—something that requires institutional commitment and sustained professional educational development:

RECOMMENDATION 2 (Support Educators): Empower programming educators to foster these competences through targeted pedagogical resources, collaboration, and sustained professional development. Broad ideals—such as *algorithmic* or *computational thinking*—should be introduced only as *complementary learning goals* once discipline-grounded competences are firmly established, as these appear distant from educator terminology and focus.

A few researchers already support educators in this direction [9], [48]. Their work, along with initiatives by Guzdial & Evrard in the Liberal Arts & Sciences [37], [49] and by Newland in Astronomy education [50], illustrates a growing interest in contextualized computing education. Future work should continue connecting educator viewpoints with institutional logics to broaden participation and deepen understanding across disciplines.

REFERENCES

- [1] R. Lister and J. Leaney, "Introductory programming, criterion-referencing, and bloom," in *Proceedings of the 34th SIGCSE technical symposium on Computer science education*, 2003, pp. 143–147.

- [2] J. M. Wing, "Computational thinking," *Communications of the ACM*, vol. 49, no. 3, pp. 33–35, 2006.
- [3] P. J. Denning and M. Tedre, *Computational thinking*. Mit Press, 2019.
- [4] —, "Computational thinking: A disciplinary perspective," *Informatics in Education*, vol. 20, no. 3, p. 361, 2021.
- [5] M. E. Caspersen, O. S. Iversen, M. Nielsen, A. Hjorth, and L. H. Musaeus, "Computational thinking," *Dolin, G. Holten Ingerslev & Hanne Sparholt Jørgensen (Red.), Gymnasiepædagogik. En grundbog*. København: Hans Reitzels, pp. 470–478, 2017.
- [6] T. Crick and S. Sentance, "Computing at school: stimulating computing education in the UK," in *Proc. of the 11th Koli Calling International Conference on Computing Education Research*, 2011, pp. 122–123.
- [7] M. Apiola, S. López-Pernas, and M. Saqr, *Past, Present and Future of Computing Education Research: A Global Perspective*. Springer Nature, 2023.
- [8] L. Benton, C. Hoyles, I. Kalas, and R. Noss, "Bridging primary programming and mathematics: Some findings of design research in england," *Digital Experiences in Mathematics Education*, vol. 3, pp. 115–138, 2017.
- [9] P. Mertala, "Teachers' beliefs about technology integration in early childhood education: A meta-ethnographical synthesis of qualitative research," *Computers in Human Behavior*, vol. 101, pp. 334–349, 2019.
- [10] S. Vogel, R. Santo, and D. Ching, "Visions of computer science education: Unpacking arguments for and projected impacts of cs4all initiatives," in *Proceedings of the 2017 ACM SIGCSE technical symposium on computer science education*, 2017, pp. 609–614.
- [11] S. Sentance and A. Csizmadia, "Computing in the curriculum: Challenges and strategies from a teacher's perspective," *Education and information technologies*, vol. 22, pp. 469–495, 2017.
- [12] A. Yadav, S. Gretter, S. Hambruch, and P. Sands, "Expanding computer science education in schools: understanding teacher experiences and challenges," *Computer science education*, vol. 26, no. 4, 2016.
- [13] K. A. Bingimlas, "Barriers to the successful integration of ict in teaching and learning environments: A review of the literature," *Eurasia Journal of Mathematics, science and technology education*, vol. 5, no. 3, pp. 235–245, 2009.
- [14] L. Greifenstein, I. Graßl, and G. Fraser, "Challenging but full of opportunities: Teachers' perspectives on programming in primary schools," in *Proceedings of the 21st Koli Calling International Conference on Computing Education Research*, 2021, pp. 1–10.
- [15] M. Saeli, J. Perrenet, W. M. Jochems, and B. Zwaneveld, "Teaching programming in secondary school: A pedagogical content knowledge perspective," *Informatics in education*, vol. 10, no. 1, pp. 73–88, 2011.
- [16] T. Korhonen, L. Salo, N. Laakso, A. Seitamaa, K. Sormunen, M. Kukkonen, and H. Forsström, "Finnish teachers as adopters of educational innovation: perceptions of programming as a new part of the curriculum," *Computer science education*, vol. 33, no. 1, pp. 94–116, 2023.
- [17] R. P. Medeiros, G. L. Ramalho, and T. P. Falcão, "A systematic literature review on teaching and learning introductory programming in higher education," *IEEE Transactions on Education*, vol. 62, no. 2, pp. 77–90, 2018.
- [18] P. Mertala, "Wag the dog—the nature and foundations of preschool educators' positive ict pedagogical beliefs," *Computers in Human Behavior*, vol. 69, pp. 197–206, 2017.
- [19] —, "Digital technologies in early childhood education—a frame analysis of preservice teachers' perceptions," *Early Child Development and Care*, vol. 189, no. 8, pp. 1228–1241, 2019.
- [20] R. Pörn, K. Hemmi, and P. Kallio-Kujala, "Inspiring or confusing—a study of finnish 1-6 teachers' relation to teaching programming," *LUMAT: International Journal on Math, Science and Technology Education*, vol. 9, no. 1, pp. 366–396, 2021.
- [21] S. M. Nicolajsen, S. Nielsen, L. M. Carlsen, and C. Brabrand, "Programming education across disciplines: a nationwide study of danish higher education," *Higher Education*, pp. 1–27, 2024.
- [22] S. A. Papert, *Mindstorms: Children, computers, and powerful ideas*. Basic books, 1980.
- [23] A. A. DiSessa, *Changing minds: Computers, learning, and literacy*. Mit Press, 2000.
- [24] —, "Computational literacy and "the big picture" concerning computers in mathematics education," *Mathematical thinking and learning*, vol. 20, no. 1, pp. 3–31, 2018.
- [25] J. Voogt, P. Fisser, J. Good, P. Mishra, and A. Yadav, "Computational thinking in compulsory education: Towards an agenda for research and practice," *Education and information technologies*, vol. 20, 2015.
- [26] D. Hemmendinger, "A plea for modesty," *Acm Inroads*, vol. 1, no. 2, pp. 4–7, 2010.
- [27] P. Curzon and P. W. McOwan, *The power of computational thinking: Games, magic and puzzles to help you become a computational thinker*. World Scientific, 2017.
- [28] R. S. G. Britain, *Shut down or restart?: The way forward for computing in UK schools*. Royal Society, 2012.
- [29] A. Yadav, H. Hong, and C. Stephenson, "Computational thinking for all: Pedagogical approaches to embedding 21st century problem solving in k-12 classrooms," *TechTrends*, vol. 60, pp. 565–568, 2016.
- [30] N. R. Council et al., *Report of a workshop on the scope and nature of computational thinking*. National Academies Press, 2010.
- [31] A. Pérez-Escoda and M. J. Rodríguez-Conde, "Digital literacy and digital competences in the educational evaluation: Usa and ica contexts," in *Proceedings of the 3rd international conference on technological ecosystems for enhancing multicultural*, 2015, pp. 355–360.
- [32] A. Juškevičienė and V. Dagienė, "Computational thinking relationship with digital competence," *Informatics in Education*, vol. 17, no. 2, pp. 265–284, 2018.
- [33] L. Ilomäki, A. Kantosalo, M. Lakkala et al., "What is digital competence," *Linked portal*, pp. 1–12, 2011.
- [34] A. J. Ko, A. Oleson, N. Ryan, Y. Register, B. Xie, M. Tari, M. Davidson, S. Druga, and D. Loksa, "It is time for more critical cs education," *Communications of the ACM*, vol. 63, no. 11, pp. 31–33, 2020.
- [35] P. Mishra and A. Yadav, "Of art and algorithms: Rethinking technology & creativity in the 21st century," pp. 10–14, 2013.
- [36] S. Grover and R. Pea, "Computational thinking in K–12: A review of the state of the field," *Educational researcher*, vol. 42, no. 1, 2013.
- [37] G. Evrard and M. Guzdial, "Identifying the computing education needs of liberal arts sciences students," in *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*, 2023.
- [38] S. M. Nicolajsen, M. Pischetola, P. Grabarczyk, and C. Brabrand, "Three+ 1 perspectives on computational thinking," in *Proceedings of the 21st Koli Calling International Conference on Computing Education Research*, 2021, pp. 1–11.
- [39] J. Rowley, "Designing and using research questionnaires," *Management research review*, 2014.
- [40] N. Gisev, J. S. Bell, and T. F. Chen, "Interrater agreement and interrater reliability: key concepts, approaches, and applications," *Research in Social and Administrative Pharmacy*, vol. 9, no. 3, pp. 330–338, 2013.
- [41] Eurostat, "ICT specialists in employment," 2022. [Online]. Available: https://ec.europa.eu/eurostat/statistics-explained/index.php/ICT_specialists_in_employment
- [42] Danmarks Statistik, "Uddannelsesregister," 2016, accessed on 21 05, 2024. [Online]. Available: <https://www.dst.dk/Site/Dst/SingleFiles/GetArchiveFile.aspx?fi=uddannelse&fo=vejluddreg--pdf&ext=%7B2%7D>
- [43] M. Shaw, "Myths and mythconceptions: What does it mean to be a programming language, anyhow?" *Proceedings of the ACM on Programming Languages*, vol. 4, no. HOPL, pp. 1–44, 2022.
- [44] F. Kalelioglu, Y. Gulbahar, and V. Kukul, "A framework for computational thinking based on a systematic research review," 2016.
- [45] H. İ. Haseski, U. İlic, and U. Tuğtekin, "Defining a new 21st century skill-computational thinking: Concepts and trends," *International Education Studies*, 2018.
- [46] Y. Chevallard, "Introducing the anthropological theory of the didactic: An attempt at a principled approach," *Hiroshima Journal of Mathematics Education*, vol. 12, pp. 71–114, 2019.
- [47] M. Bosch and J. Gascón, "Anthropological theory of the didactic," in *Encyclopedia of Mathematics Education*, S. Lerman, Ed. Cham: Springer, 2020, pp. 44–49.
- [48] T. O. B. Odden, D. W. Silvia, and A. Malthe-Sørensen, "Using computational essays to foster disciplinary epistemic agency in undergraduate science," *Journal of Research in Science Teaching*, vol. 60, no. 5, pp. 937–977, 2023.
- [49] E. August, M. J. Guzdial, E. Bruch, N. Henriksen, A. Kamil, R. Lewis, A. Ostling, S. Schoenbeck, and M. R. Yergeau, "Report of the isa computing education task force."
- [50] J. J. Newland, "Integrating data science pedagogy into astronomy courses background."